Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ РОБОТИ № 5

з дисципліни «Професійний Python»

на тему: «Обробка помилок, файли та серіалізація»

Виконала:
студентка 3 курсу
групи ФеП-33
Наборська Дарія
Русланівна

Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

Тема роботи

Лабораторна робота № 5. Обробка помилок, файли та серіалізація.

Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

Номер індивідуального варіанта

Варіант 9. Імпорт та експорт спортивних результатів.

Постановка задачі

Створити надійний модуль імпорту, експорту та конфігурації спортивних результатів. Поля: athlete, sport, result, category, timestamp. Перевіряти спортсмена, sport/category, числовий result і timestamp; правила допустимого result задавати через configuration. Реалізувати tolerant import, окремий error report, JSON export, statistics і streaming processing.

Модуль працює з Path, читає CSV, підтримує JSON, використовує YAML configuration, streaming input, validation, власну exception hierarchy, exception chaining, with і власні context managers, logging, strict/tolerant режими, статистику valid/invalid, export з atomic output, обробку file I/O errors і коректне завершення при critical errors.

Завдання підвищеної складності

Для максимальної оцінки потрібно щонайменше три додаткові можливості. Реалізовано:

- **CSV, JSON, JSON Lines і YAML input через єдиний interface** — Importer Protocol і CsvImporter, JsonImporter, JsonLinesImporter, YamlImporter;
- **exporter Protocol та importer Protocol**;
- **автоматичний вибір format за extension** — importer_for, exporter_for;
- **окремий error CSV** — output/errors.csv;
- **configurable logging і log rotation** — RotatingFileHandler з max_bytes/backup_count із config;
- **nested configuration, environment-variable override, configuration schema version** — SPORT_RESULTS_MODE, SPORT_RESULTS_LOG_LEVEL, schema_version;
- **atomic export і резервне копіювання existing output** — atomic_write(path, backup=True) створює .bak;

- **streaming JSON Lines, progress statistics, export summary report**;
- **context manager для transaction-like file operation** — FileTransaction;
- **validation policy object** — ValidationPolicy;
- **retry для тимчасового file access error** — retry() для PermissionError;
- **compression через gzip і custom JSON encoder** — *.json.gz, ResultEncoder для datetime.

Репозиторій: https://github.com/dashanaborska/python_lab, каталог Lab_05 (звіт — у Lab_05/report/).

Input data schema

Поле	Тип	Правило валідації
athlete	str	не порожнє, без цифр, пробіли нормалізуються
sport	str	із переліку processing.sports у config.yaml
result	float	число (кома або крапка) у межах processing.result.min…max
category	str	із переліку processing.categories
timestamp	datetime	формат ISO 8601 (YYYY-MM-DDTHH:MM:SS)

```python
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any


@dataclass(frozen=True, slots=True)
class SportResult:
    athlete: str
    sport: str
    result: float
    category: str
    timestamp: datetime

    def to_dict(self) -> dict[str, Any]:
        return {
            "athlete": self.athlete,
            "sport": self.sport,
            "result": self.result,
            "category": self.category,
            "timestamp": self.timestamp,
        }


@dataclass(frozen=True, slots=True)
```

```
class InvalidRecord:
    line_number: int
    field: str
    reason: str
    raw: str


@dataclass
class ImportStats:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    exported: int = 0
    by_field: dict[str, int] = field(default_factory=dict)

    def register_invalid(self, field_name: str) -> None:
        self.invalid += 1
        self.by_field[field_name] = self.by_field.get(field_name, 0) + 1

    def to_dict(self) -> dict[str, Any]:
        return {
            "total": self.total,
            "valid": self.valid,
            "invalid": self.invalid,
            "exported": self.exported,
            "invalid_by_field": dict(sorted(self.by_field.items())),
        }
```

Приклад CSV

Файл data/results.csv містить 16 записів, з них 6 навмисно некоректні: нечисловий result, result = 124, невідома категорія, порожнє ім'я, невідомий вид спорту, неправильний timestamp; один запис має зайві пробіли в імені.

```csv
athlete,sport,result,category,timestamp
Олена Ковальчук,Плавання,87.5,КМС,2026-03-01T09:00:00
Максим Бондар,Легка атлетика,92.3,МС,2026-03-01T09:05:00
Ірина Мельник,Плавання,78.4,I розряд,2026-03-01T09:10:00
Андрій Шевченко,Бокс,81.0,КМС,2026-03-01T09:15:00
Софія Ткачук,Легка атлетика,95.1,МСМК,2026-03-01T09:20:00
Тарас Новак,Бокс,wrong,МС,2026-03-01T09:25:00
Марія Бойко,Плавання,90.2,МС,2026-03-01T09:30:00
  Дмитро Савчук ,Велоспорт,73.8,II розряд,2026-03-01T09:35:00
Юлія Кравець,Велоспорт,124.0,КМС,2026-03-01T09:40:00
Богдан Лисенко,Легка атлетика,69.5,майстер,2026-03-01T09:45:00
,Бокс,86.4,КМС,2026-03-01T09:50:00
Олег Петренко,Шахи,91.7,МС,2026-03-01T09:55:00
Катерина Гнатюк,Бокс,86.4,КМС,01.03.2026 10:00
Наталія Мороз,Плавання,88.9,МС,2026-03-01T10:05:00
Віктор Руденко,Бокс,79.6,I розряд,2026-03-01T10:10:00
Анна Литвин,Легка атлетика,84.2,КМС,2026-03-01T10:15:00
```

Структура проєкту

```
None
```

```
python_lab/
└── Lab_05/
    ├── config.yaml
    ├── pyproject.toml
    ├── README.md
    ├── data/
    │   ├── results.csv
    │   └── generated/          (створюється експериментами)
    ├── output/                 (results.json, errors.csv, summary.json)
    ├── logs/                   (application.log з ротацією)
    ├── report/
    │   └── Lab_05_Naborska.pdf
    ├── src/
    │   └── sport_results/
    │       ├── __init__.py
    │       ├── config.py           завантаження і валідація config.yaml
    │       ├── exceptions.py       exception hierarchy
    │       ├── experiments.py      5 експериментів
    │       ├── exporters.py        JSON / JSON Lines / errors / summary
    │       ├── files.py            atomic_write, FileTransaction, retry
    │       ├── generator.py        генерація CSV для експериментів
    │       ├── logging_config.py   налаштування logging
    │       ├── main.py             точка входу, коди завершення
    │       ├── models.py           SportResult, InvalidRecord, ImportStats
    │       ├── readers.py          importers, streaming reader
    │       ├── services.py         ImportService (strict / tolerant)
    │       └── validators.py       ValidationPolicy
    └── tests/
        └── test_importer.py
```

Exception hierarchy

None

```
ApplicationError
├── ConfigurationError
├── DataError
│   ├── DataImportError
│   │   └── UnsupportedFormatError
│   └── RecordValidationError (line_number, field)
└── ExportError
```

Python

```
class ApplicationError(Exception):
    pass


class ConfigurationError(ApplicationError):
    pass


class DataError(ApplicationError):
    pass


class DataImportError(DataError):
    pass


class UnsupportedFormatError(DataImportError):
```

```
    pass


class RecordValidationError(DataError):
    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.message = message
        self.line_number = line_number
        self.field = field

    def __str__(self) -> str:
        location = f"рядок {self.line_number}" if self.line_number else "?"
        field = f", поле {self.field}" if self.field else ""
        return f"{location}{field}: {self.message}"


class ExportError(ApplicationError):
    pass
```

Пояснення custom exceptions

- **ApplicationError** — базовий виняток застосунку: main може перехопити всі очікувані помилки одним except, не ховаючи програмні помилки (TypeError тощо).
- **ConfigurationError** — проблема конфігурації (відсутній файл, некоректний YAML, неправильні значення); застосунок завершується з кодом 2 ще до обробки даних (fail-fast).
- **DataImportError / UnsupportedFormatError** — вхідні дані неможливо прочитати: файла немає, формат не підтримується, некоректний JSON, файл не в UTF-8; також зупинка в strict mode.
- **RecordValidationError** — некоректний окремий запис; зберігає номер рядка і поле, тому в tolerant mode запис потрапляє в error report з точною причиною.
- **ExportError** — неможливо записати результат (немає доступу до каталогу, збій під час export).

Приклад exception chaining

Python

```
def parse_result(self, raw: str, line: int) -> float:
    try:
        value = float(raw.strip().replace(",", "."))
    except ValueError as error:
        raise RecordValidationError(
            f"result не є числом: {raw!r}",
```

```
            line_number=line,
            field="result",
        ) from error
```

raise ... from error зберігає оригінальний ValueError у __cause__. У strict mode ланцюжок триває: ValueError → RecordValidationError → DataImportError. У traceback видно і технічну причину (could not convert string to float), і бізнес-контекст (рядок, поле), що спрощує налагодження. Так само ConfigurationError зберігає FileNotFoundError або yaml.YAMLError. Результат експерименту — на рисунку 8 (розділ 3).

Код configuration loader

Python

```
import os
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import yaml

from sport_results.exceptions import ConfigurationError

SUPPORTED_SCHEMA = 1
MODES = ("strict", "tolerant")
LEVELS = ("DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL")
ENV_PREFIX = "SPORT_RESULTS_"


@dataclass(frozen=True)
class ResultRange:
    minimum: float
    maximum: float


@dataclass(frozen=True)
class ProcessingConfig:
    mode: str
    result: ResultRange
    sports: frozenset[str]
    categories: frozenset[str]
    progress_every: int

    @property
    def strict(self) -> bool:
        return self.mode == "strict"


@dataclass(frozen=True)
class OutputConfig:
    path: Path
    errors_path: Path
    summary_path: Path
    backup: bool


@dataclass(frozen=True)
```

```
class LoggingConfig:
    level: str
    path: Path
    max_bytes: int
    backup_count: int


@dataclass(frozen=True)
class AppConfig:
    input_path: Path
    output: OutputConfig
    processing: ProcessingConfig
    logging: LoggingConfig


def read_yaml(path: Path) -> dict[str, Any]:
    try:
        with path.open("r", encoding="utf-8") as file:
            data = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(
            f"Файл конфігурації {path} не знайдено"
        ) from error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"Некоректний YAML у {path}") from error
    if not isinstance(data, dict):
        raise ConfigurationError("Конфігурація має бути YAML-словником")
    return data


def section(data: dict[str, Any], name: str) -> dict[str, Any]:
    value = data.get(name)
    if not isinstance(value, dict):
        raise ConfigurationError(f"Відсутня секція '{name}'")
    return value


def require(data: dict[str, Any], key: str, where: str) -> Any:
    if key not in data:
        raise ConfigurationError(f"Відсутній ключ '{where}.{key}'")
    return data[key]


def apply_env(data: dict[str, Any], env: dict[str, str]) -> dict[str, Any]:
    processing = dict(data.get("processing", {}))
    logging = dict(data.get("logging", {}))
    if f"{ENV_PREFIX}MODE" in env:
        processing["mode"] = env[f"{ENV_PREFIX}MODE"]
    if f"{ENV_PREFIX}LOG_LEVEL" in env:
        logging["level"] = env[f"{ENV_PREFIX}LOG_LEVEL"]
    return {**data, "processing": processing, "logging": logging}


def parse_range(raw: dict[str, Any]) -> ResultRange:
    try:
        minimum = float(require(raw, "min", "processing.result"))
        maximum = float(require(raw, "max", "processing.result"))
    except (TypeError, ValueError) as error:
        raise ConfigurationError("Межі result мають бути числами") from error
    if not 0 <= minimum < maximum <= 100:
        raise ConfigurationError(
            f"Некоректний діапазон result: [{minimum}; {maximum}]"
```

```
        )
    return ResultRange(minimum, maximum)


def parse_config(
    data: dict[str, Any],
    base: Path,
    env: dict[str, str] | None = None,
) -> AppConfig:
    data = apply_env(data, dict(os.environ) if env is None else env)
    version = data.get("schema_version")
    if version != SUPPORTED_SCHEMA:
        raise ConfigurationError(
            f"Непідтримувана schema_version: {version!r}"
        )
    source = section(data, "input")
    output = section(data, "output")
    processing = section(data, "processing")
    log = section(data, "logging")
    mode = str(processing.get("mode", "tolerant"))
    if mode not in MODES:
        raise ConfigurationError(f"Невідомий mode: {mode!r}")
    level = str(log.get("level", "INFO")).upper()
    if level not in LEVELS:
        raise ConfigurationError(f"Невідомий рівень logging: {level!r}")
    sports = require(processing, "sports", "processing")
    categories = require(processing, "categories", "processing")
    if not sports or not categories:
        raise ConfigurationError("Списки sports і categories не порожні")
    return AppConfig(
        input_path=base / str(require(source, "path", "input")),
        output=OutputConfig(
            path=base / str(require(output, "path", "output")),
            errors_path=base / str(output.get("errors_path", "errors.csv")),
            summary_path=base
            / str(output.get("summary_path", "summary.json")),
            backup=bool(output.get("backup", False)),
        ),
        processing=ProcessingConfig(
            mode=mode,
            result=parse_range(section(processing, "result")),
            sports=frozenset(str(item) for item in sports),
            categories=frozenset(str(item) for item in categories),
            progress_every=int(processing.get("progress_every", 1000)),
        ),
        logging=LoggingConfig(
            level=level,
            path=base / str(log.get("path", "logs/application.log")),
            max_bytes=int(log.get("max_bytes", 1_048_576)),
            backup_count=int(log.get("backup_count", 3)),
        ),
    )


def load_config(path: Path, env: dict[str, str] | None = None) -> AppConfig:
    return parse_config(read_yaml(path), path.parent, env)
```

Вміст config.yaml

```
YAML
```

```yaml
schema_version: 1

input:
  path: data/results.csv

output:
  path: output/results.json
  errors_path: output/errors.csv
  summary_path: output/summary.json
  backup: true

processing:
  mode: tolerant
  progress_every: 10
  result:
    min: 0
    max: 100
  sports:
    - Плавання
    - Бокс
    - Легка атлетика
    - Велоспорт
  categories:
    - МСМК
    - МС
    - КМС
    - I розряд
    - II розряд
    - III розряд

logging:
  level: INFO
  path: logs/application.log
  max_bytes: 1048576
  backup_count: 3
```

Validation configuration

- **schema_version** — підтримується лише версія 1, інакше ConfigurationError.
- **секції input, output, processing, logging** — обов'язкові словники; ключі input.path, output.path, processing.sports, processing.categories, processing.result.min/max обов'язкові.
- **processing.result** — числа, $0 \le \text{min} < \text{max} \le 100$.
- **processing.mode** — strict або tolerant; logging.level — один із стандартних рівнів.
- **override** — змінні середовища SPORT_RESULTS_MODE і SPORT_RESULTS_LOG_LEVEL та аргумент --mode мають пріоритет над файлом.
- **yaml.safe_load** — не дозволяє створювати довільні Python-об'єкти з YAML.

Logging configuration

```python
import logging
from logging.handlers import RotatingFileHandler

from sport_results.config import LoggingConfig

FORMAT = "%(asctime)s %(levelname)-8s %(module)s: %(message)s"


def setup_logging(config: LoggingConfig, console: bool = True) -> None:
    config.path.parent.mkdir(parents=True, exist_ok=True)
    root = logging.getLogger("sport_results")
    root.handlers.clear()
    root.setLevel(config.level)
    root.propagate = False
    file_handler = RotatingFileHandler(
        config.path,
        maxBytes=config.max_bytes,
        backupCount=config.backup_count,
        encoding="utf-8",
    )
    file_handler.setFormatter(logging.Formatter(FORMAT))
    root.addHandler(file_handler)
    if console:
        stream = logging.StreamHandler()
        stream.setLevel(logging.WARNING)
        stream.setFormatter(logging.Formatter("%(levelname)s: %(message)s"))
        root.addHandler(stream)
```

Логер sport_results пише всі події від рівня з конфігурації у файл з ротацією (1 MB, 3 резервні копії), а в консоль — лише WARNING і вище. Модулі отримують логер через logging.getLogger(__name__). У логи не записуються зайві дані — лише номер рядка, поле та причина.

Приклад log file

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> Get-Content logs\application.log -Tail 16
2026-10-22 19:54:52,103 INFO     services: Старт імпорту data\results.csv (mode=tolerant)
2026-10-22 19:54:52,104 WARNING  services: Пропущено запис: рядок 7, поле result: result не є числом: 'wrong'
2026-10-22 19:54:52,104 WARNING  services: Пропущено запис: рядок 10, поле result: result 124.0 поза [0.0; 100.0]
2026-10-22 19:54:52,105 WARNING  services: Пропущено запис: рядок 11, поле category: невідома категорія 'майстер'
2026-10-22 19:54:52,105 WARNING  services: Пропущено запис: рядок 12, поле athlete: порожнє значення
2026-10-22 19:54:52,105 WARNING  services: Пропущено запис: рядок 13, поле sport: невідомий вид спорту 'Шахи'
2026-10-22 19:54:52,105 WARNING  services: Пропущено запис: рядок 14, поле timestamp: timestamp не у форматі ISO 8601: '01.03.2026 10:00'
2026-10-22 19:54:52,107 INFO     files: Атомарно записано output\results.json
2026-10-22 19:54:52,107 INFO     files: Атомарно записано output\errors.csv
2026-10-22 19:54:52,108 INFO     files: Атомарно записано output\summary.json
2026-10-22 19:54:52,108 INFO     services: Завершено: total=16, valid=10, invalid=6, exported=10
2026-10-22 19:54:52,250 INFO     services: Старт імпорту data\results.csv (mode=strict)
2026-10-22 19:54:52,251 ERROR    services: Strict mode: рядок 7, поле result: result не є числом: 'wrong'
2026-10-22 19:54:52,252 WARNING  files: Запис output\results.json скасовано, тимчасовий файл видалено
2026-10-22 19:54:52,253 CRITICAL main: Імпорт не виконано: Імпорт зупинено у strict mode (рядок 7, поле result: result не є числом: 'wrong')
```

Рисунок 1 — Фрагмент logs/application.log після tolerant і strict запуску

Streaming reader

```python
import csv
import json
```

```
from collections.abc import Iterator
from pathlib import Path
from typing import Protocol

import yaml

from sport_results.exceptions import DataImportError, UnsupportedFormatError

Row = dict[str, str]
NumberedRow = tuple[int, Row]


class Importer(Protocol):
    def read(self, path: Path) -> Iterator[NumberedRow]: ...


def as_row(item: object, line: int) -> Row:
    if not isinstance(item, dict):
        raise DataImportError(f"Запис {line} не є об'єктом")
    return {str(key): "" if value is None else str(value)
            for key, value in item.items()}


class CsvImporter:
    def read(self, path: Path) -> Iterator[NumberedRow]:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for row in reader:
                yield reader.line_num, {
                    key: value or "" for key, value in row.items() if key
                }


class JsonLinesImporter:
    def read(self, path: Path) -> Iterator[NumberedRow]:
        with path.open("r", encoding="utf-8") as file:
            for number, line in enumerate(file, start=1):
                if not line.strip():
                    continue
                try:
                    item = json.loads(line)
                except json.JSONDecodeError as error:
                    raise DataImportError(
                        f"Некоректний JSON у рядку {number}"
                    ) from error
                yield number, as_row(item, number)


class JsonImporter:
    def read(self, path: Path) -> Iterator[NumberedRow]:
        try:
            with path.open("r", encoding="utf-8") as file:
                items = json.load(file)
        except json.JSONDecodeError as error:
            raise DataImportError(f"Некоректний JSON у {path}") from error
        if not isinstance(items, list):
            raise DataImportError("JSON має містити масив записів")
        for number, item in enumerate(items, start=1):
            yield number, as_row(item, number)


class YamlImporter:
```

```
    def read(self, path: Path) -> Iterator[NumberedRow]:
        try:
            with path.open("r", encoding="utf-8") as file:
                items = yaml.safe_load(file) or []
        except yaml.YAMLError as error:
            raise DataImportError(f"Некоректний YAML у {path}") from error
        if not isinstance(items, list):
            raise DataImportError("YAML має містити список записів")
        for number, item in enumerate(items, start=1):
            yield number, as_row(item, number)


IMPORTERS: dict[str, Importer] = {
    ".csv": CsvImporter(),
    ".jsonl": JsonLinesImporter(),
    ".json": JsonImporter(),
    ".yaml": YamlImporter(),
    ".yml": YamlImporter(),
}


def importer_for(path: Path) -> Importer:
    try:
        return IMPORTERS[path.suffix.lower()]
    except KeyError:
        raise UnsupportedFormatError(
            f"Непідтримуваний формат {path.suffix!r}"
        ) from None


def read_records(path: Path) -> Iterator[NumberedRow]:
    if not path.is_file():
        raise DataImportError(f"Вхідний файл {path} не знайдено")
    try:
        yield from importer_for(path).read(path)
    except UnicodeDecodeError as error:
        raise DataImportError(f"Файл {path} не в UTF-8") from error
    except OSError as error:
        raise DataImportError(f"Помилка читання {path}") from error


def eager_import(path: Path) -> list[Row]:
    with path.open(encoding="utf-8", newline="") as file:
        return list(csv.DictReader(file))


def streaming_import(path: Path) -> Iterator[Row]:
    with path.open(encoding="utf-8", newline="") as file:
        yield from csv.DictReader(file)
```

CsvImporter і JsonLinesImporter читають файл по одному запису через with і yield — у пам'яті одночасно лише поточний рядок. JSON-масив і YAML за своєю природою завантажуються повністю, тому їх використовують для невеликих файлів. read_records перевіряє наявність файлу і перетворює UnicodeDecodeError та OSError на DataImportError із chaining.

Validation code

```python
from dataclasses import dataclass
from datetime import datetime

from sport_results.config import ProcessingConfig
from sport_results.exceptions import RecordValidationError
from sport_results.models import SportResult
from sport_results.readers import Row

FIELDS = ("athlete", "sport", "result", "category", "timestamp")


@dataclass(frozen=True)
class ValidationPolicy:
    minimum: float
    maximum: float
    sports: frozenset[str]
    categories: frozenset[str]

    @classmethod
    def from_config(cls, config: ProcessingConfig) -> "ValidationPolicy":
        return cls(
            config.result.minimum,
            config.result.maximum,
            config.sports,
            config.categories,
        )

    def validate(self, row: Row, line: int) -> SportResult:
        for name in FIELDS:
            if not row.get(name, "").strip():
                raise RecordValidationError(
                    "порожнє значення", line_number=line, field=name
                )
        athlete = " ".join(row["athlete"].split())
        if any(char.isdigit() for char in athlete):
            raise RecordValidationError(
                f"ім'я містить цифри: {athlete!r}",
                line_number=line,
                field="athlete",
            )
        sport = row["sport"].strip()
        if sport not in self.sports:
            raise RecordValidationError(
                f"невідомий вид спорту {sport!r}",
                line_number=line,
                field="sport",
            )
        category = row["category"].strip()
        if category not in self.categories:
            raise RecordValidationError(
                f"невідома категорія {category!r}",
                line_number=line,
                field="category",
            )
        return SportResult(
            athlete=athlete,
            sport=sport,
            result=self.parse_result(row["result"], line),
```

```
            category=category,
            timestamp=self.parse_timestamp(row["timestamp"], line),
        )

    def parse_result(self, raw: str, line: int) -> float:
        try:
            value = float(raw.strip().replace(",", "."))
        except ValueError as error:
            raise RecordValidationError(
                f"result не є числом: {raw!r}",
                line_number=line,
                field="result",
            ) from error
        if not self.minimum <= value <= self.maximum:
            raise RecordValidationError(
                f"result {value} поза [{self.minimum}; {self.maximum}]",
                line_number=line,
                field="result",
            )
        return value

    def parse_timestamp(self, raw: str, line: int) -> datetime:
        try:
            return datetime.fromisoformat(raw.strip())
        except ValueError as error:
            raise RecordValidationError(
                f"timestamp не у форматі ISO 8601: {raw!r}",
                line_number=line,
                field="timestamp",
            ) from error
```

Strict та tolerant processing

Python

```
import logging
from collections.abc import Iterator
from datetime import datetime
from pathlib import Path

from sport_results.config import AppConfig
from sport_results.exceptions import DataImportError, RecordValidationError
from sport_results.exporters import exporter_for, write_errors, write_summary
from sport_results.models import ImportStats, InvalidRecord, SportResult
from sport_results.readers import read_records
from sport_results.validators import ValidationPolicy

logger = logging.getLogger(__name__)


class ImportService:
    def __init__(self, config: AppConfig) -> None:
        self.config = config
        self.policy = ValidationPolicy.from_config(config.processing)
        self.stats = ImportStats()
        self.errors: list[InvalidRecord] = []

    def records(self, path: Path) -> Iterator[SportResult]:
        progress = self.config.processing.progress_every
        for line, row in read_records(path):
            self.stats.total += 1
            try:
                record = self.policy.validate(row, line)
            except RecordValidationError as error:
                self.handle_invalid(error, row)
                continue
            self.stats.valid += 1
            if progress and self.stats.total % progress == 0:
                logger.info(
                    "Прогрес: %d оброблено, %d valid, %d invalid",
                    self.stats.total, self.stats.valid, self.stats.invalid,
                )
            yield record

    def handle_invalid(
        self,
        error: RecordValidationError,
        row: dict[str, str],
    ) -> None:
        field = error.field or "record"
        self.stats.register_invalid(field)
        if self.config.processing.strict:
            logger.error("Strict mode: %s", error)
            raise DataImportError(
                f"Імпорт зупинено у strict mode ({error})"
            ) from error
        self.errors.append(
            InvalidRecord(
                error.line_number or 0, field, error.message,
                ",".join(row.values()),
            )
        )
        logger.warning("Пропущено запис: %s", error)
```

```
    def run(self) -> ImportStats:
        config = self.config
        logger.info(
            "Старт імпорту %s (mode=%s)",
            config.input_path, config.processing.mode,
        )
        exporter = exporter_for(config.output.path)
        self.stats.exported = exporter.export(
            self.records(config.input_path),
            config.output.path,
            config.output.backup,
        )
        write_errors(self.errors, config.output.errors_path)
        write_summary(self.summary(), config.output.summary_path)
        stats = self.stats
        logger.info(
            "Завершено: total=%d, valid=%d, invalid=%d, exported=%d",
            stats.total, stats.valid, stats.invalid, stats.exported,
        )
        return self.stats

    def summary(self) -> dict[str, object]:
        return {
            "generated_at": datetime.now().replace(microsecond=0),
            "input": str(self.config.input_path.name),
            "mode": self.config.processing.mode,
            "statistics": self.stats.to_dict(),
        }
```

- **Strict processing** — перший некоректний запис логується як ERROR, і піднімається DataImportError (from RecordValidationError). Оскільки export виконується всередині atomic_write, тимчасовий файл видаляється, а попередній output залишається без змін; застосунок завершується з кодом 1.
- **Tolerant processing** — некоректні записи пропускаються, кожен логується як WARNING і додається до error report; валідні записи експортуються, застосунок завершується з кодом 0.

Import statistics

ImportStats рахує total, valid, invalid, exported та invalid за полями. Прогрес логується кожні progress_every записів, а підсумок записується в output/summary.json.

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> Get-Content output\summary.json
{
  "generated_at": "2026-10-22T19:54:52",
  "input": "results.csv",
  "mode": "tolerant",
  "statistics": {
    "total": 16,
    "valid": 10,
    "invalid": 6,
    "exported": 10,
    "invalid_by_field": {
      "athlete": 1,
      "category": 1,
      "result": 2,
      "sport": 1,
      "timestamp": 1
    }
  }
}
```

Рисунок 2 — Export summary report

JSON serialization

Python

```python
class ResultEncoder(json.JSONEncoder):
    def default(self, value: Any) -> Any:
        if isinstance(value, datetime):
            return value.isoformat()
        return super().default(value)


class JsonExporter:
    def export(
        self,
        records: Iterable[SportResult],
        path: Path,
        backup: bool = False,
    ) -> int:
        count = 0
        with atomic_write(path, backup) as file:
            file.write("[\n")
            for record in records:
                if count:
                    file.write(",\n")
                file.write("  ")
                json.dump(
                    record.to_dict(), file, ensure_ascii=False,
                    cls=ResultEncoder,
                )
                count += 1
            file.write("\n]\n")
        return count
```

JSON масив формується потоково: записи пишуться у файл по одному, тож увесь список не тримається в пам'яті. ResultEncoder серіалізує datetime у ISO-рядок, ensure_ascii=False зберігає кирилицю.

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> Get-Content output\results.json -Head 4
[
  {"athlete": "Олена Ковальчук", "sport": "Плавання", "result": 87.5, "category": "КМС", "timestamp": "2026-03-01T09:00:00"},
  {"athlete": "Максим Бондар", "sport": "Легка атлетика", "result": 92.3, "category": "МС", "timestamp": "2026-03-01T09:05:00"},
  {"athlete": "Ірина Мельник", "sport": "Плавання", "result": 78.4, "category": "І розряд", "timestamp": "2026-03-01T09:10:00"},
```

Рисунок 3 — Початок output/results.json

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> Get-Content output\errors.csv
line,field,reason,raw
7,result,result не є числом: 'wrong',"Тарас Новак,Бокс,wrong,МС,2026-03-01T09:25:00"
10,result,result 124.0 поза [0.0; 100.0],"Юлія Кравець,Велоспорт,124.0,КМС,2026-03-01T09:40:00"
11,category,невідома категорія 'майстер',"Богдан Лисенко,Легка атлетика,69.5,майстер,2026-03-01T09:45:00"
12,athlete,порожнє значення,",Бокс,86.4,КМС,2026-03-01T09:50:00"
13,sport,невідомий вид спорту 'Шахи',"Олег Петренко,Шахи,91.7,МС,2026-03-01T09:55:00"
14,timestamp,timestamp не у форматі ISO 8601: '01.03.2026 10:00',"Катерина Гнатюк,Бокс,86.4,КМС,01.03.2026 10:00"
```

Рисунок 4 — Окремий error report output/errors.csv

Atomic export

```python
@contextmanager
def atomic_write(path: Path, backup: bool = False) -> Iterator[IO[str]]:
    try:
        path.parent.mkdir(parents=True, exist_ok=True)
    except OSError as error:
        raise ExportError(f"Не вдалося створити {path.parent}") from error
    descriptor, temporary = tempfile.mkstemp(
        prefix=f".{path.name}.", suffix=".tmp", dir=path.parent
    )
    temp_path = Path(temporary)
    os.close(descriptor)
    try:
        if path.suffix == ".gz":
            stream: IO[str] = gzip.open(temp_path, "wt", encoding="utf-8")
        else:
            stream = temp_path.open("w", encoding="utf-8", newline="")
        with stream:
            yield stream
        if backup and path.exists():
            shutil.copy2(path, path.with_name(path.name + ".bak"))
        os.replace(temp_path, path)
        logger.info("Атомарно записано %s", path)
    except BaseException:
        temp_path.unlink(missing_ok=True)
        logger.warning("Запис %s скасовано, тимчасовий файл видалено", path)
        raise
```

Дані спочатку пишуться у тимчасовий файл у тому ж каталозі, і лише після успішного закриття він атомарно замінює цільовий файл через os.replace. За будь-якої помилки тимчасовий файл видаляється, а існуючий output не змінюється. Якщо backup увімкнено, перед заміною зберігається копія .bak.

Власний context manager

```python
class FileTransaction:
    def __init__(self) -> None:
        self._staged: list[tuple[Path, Path]] = []
```

```
    def stage(self, path: Path) -> Path:
        path.parent.mkdir(parents=True, exist_ok=True)
        staged = path.with_name(f".{path.name}.staged")
        self._staged.append((staged, path))
        return staged

    def __enter__(self) -> "FileTransaction":
        return self

    def __exit__(
        self,
        exc_type: type[BaseException] | None,
        exc: BaseException | None,
        traceback: object,
    ) -> Literal[False]:
        if exc_type is None:
            for staged, target in self._staged:
                os.replace(staged, target)
            logger.info("Transaction: зафіксовано %d файл(и)",
                        len(self._staged))
        else:
            for staged, _ in self._staged:
                staged.unlink(missing_ok=True)
            logger.error("Transaction: відкат через %s", exc_type.__name__)
        return False


def retry(
    action: Callable[[], T],
    attempts: int = 3,
    delay: float = 0.05,
) -> T:
    for attempt in range(1, attempts + 1):
        try:
            return action()
        except PermissionError:
            if attempt == attempts:
                raise
            logger.warning("Файл зайнятий, спроба %d з %d", attempt, attempts)
            time.sleep(delay)
    raise RuntimeError("unreachable")
```

atomic_write — generator-based context manager (contextlib.contextmanager): код до yield виконується в __enter__, після — в __exit__, а except BaseException гарантує відкат. FileTransaction — class-based context manager з __enter__/__exit__: усі staged-файли фіксуються разом лише якщо блок with завершився без помилки, інакше всі видаляються. __exit__ повертає False, тому виняток не приховується.

Screenshot execution

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> python -m pip install -e ".[dev]"
    Found existing installation: sport-results 0.5.0
    Uninstalling sport-results-0.5.0:
      Successfully uninstalled sport-results-0.5.0
  Successfully installed sport-results-0.5.0
```

Рисунок 5 — Встановлення пакета з PyYAML і dev-залежностями

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> sport-results
  WARNING: Пропущено запис: рядок 7, поле result: result не є числом: 'wrong'
  WARNING: Пропущено запис: рядок 10, поле result: result 124.0 поза [0.0; 100.0]
  WARNING: Пропущено запис: рядок 11, поле category: невідома категорія 'майстер'
  WARNING: Пропущено запис: рядок 12, поле athlete: порожнє значення
  WARNING: Пропущено запис: рядок 13, поле sport: невідомий вид спорту 'Шахи'
  WARNING: Пропущено запис: рядок 14, поле timestamp: timestamp не у форматі ISO 8601: '01.03.2026 10:00'
  Режим:        tolerant
  Оброблено:    16
  Valid:        10
  Invalid:      6
  Exported:     10
    invalid athlete   1
    invalid category  1
    invalid result    2
    invalid sport     1
    invalid timestamp 1
  JSON export:  output\results.json
  Error report: output\errors.csv
  Summary:      output\summary.json
  Log:          logs\application.log
  exit code: 0
```

Рисунок 6 — Tolerant import: warnings, статистика, exit code 0

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> sport-results --mode strict
  ERROR: Strict mode: рядок 7, поле result: result не є числом: 'wrong'
  WARNING: Запис output\results.json скасовано, тимчасовий файл видалено
  CRITICAL: Імпорт не виконано: Імпорт зупинено у strict mode (рядок 7, поле result: result не є числом: 'wrong')
  Режим:        strict
  Оброблено:    6
  Valid:        5
  Invalid:      1
  Exported:     0
    invalid result    1
  Output не змінено (atomic write скасовано)
  Log:          logs\application.log
  exit code: 1
```

Рисунок 7 — Strict import: зупинка на першій помилці, output не змінено, exit code 1

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> sport-results --config absent.yaml
  CONFIG ERROR: Файл конфігурації absent.yaml не знайдено ([Errno 2] No such file or directory: 'absent.yaml')
  exit code: 2
```

Рисунок 8 — Відсутній config.yaml: exit code 2

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> sport-results-experiment
  1. Strict vs tolerant (1000 valid + 10 invalid)
  Параметр                      Strict      Tolerant
  Оброблено                          7          1010
  Valid                              6          1000
  Invalid                            1            10
  Exported                           0          1000
  Завершення              stop, рядок 8       успішно

  2. Streaming vs eager
    Records |  Eager, s | Stream, s |  Eager MB | Stream MB
  ------------------------------------------------------------
     10,000 |     0.043 |     0.025 |      5.04 |     0.062
    100,000 |     0.313 |     0.330 |     50.18 |     0.062
    500,000 |     2.642 |     1.221 |    250.99 |     0.062

  3. Exception chaining
    Traceback (most recent call last):
    ValueError: could not convert string to float: 'wrong'
    The above exception was the direct cause of the following exception:
    Traceback (most recent call last):
    sport_results.exceptions.RecordValidationError: рядок 2, поле result: result не є числом: 'wrong'

  4. Atomic output
    ExportError перехоплено: Імітація збою під час export
    попередній output не пошкоджено: True
    валідний JSON: 10 записів
    тимчасових .tmp файлів: 0

  5. Configuration errors
    відсутній config.yaml    -> ConfigurationError: Файл конфігурації absent.yaml не знайдено [cause: FileNotFoundError]
    неправильний YAML        -> ConfigurationError: Некоректний YAML у output\experiments\broken.yaml [cause: ParserError]
    немає ключа input        -> ConfigurationError: Відсутня секція 'input' [cause: -]
    result.max = 150         -> ConfigurationError: Некоректний діапазон result: [0.0; 150.0] [cause: -]
    input file відсутній     -> DataImportError: Вхідний файл data\absent.csv не знайдено [cause: -]
    output dir недоступний   -> ExportError: Не вдалося створити output\experiments\blocker [cause: FileExistsError]
```

Рисунок 9 — Результати експериментів 1–5

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> python -m pytest -v
  ============================= test session starts ==============================
  platform win32 -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05\.venv\Scripts\python.exe
  rootdir: C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05
  configfile: pyproject.toml
  collecting ... collected 15 items

  tests\test_importer.py::test_load_config_and_env_override PASSED         [  6%]
  tests\test_importer.py::test_invalid_configuration[<lambda>0] PASSED     [ 13%]
  tests\test_importer.py::test_invalid_configuration[<lambda>1] PASSED     [ 20%]
  tests\test_importer.py::test_invalid_configuration[<lambda>2] PASSED     [ 26%]
  tests\test_importer.py::test_invalid_configuration[<lambda>3] PASSED     [ 33%]
  tests\test_importer.py::test_broken_yaml_is_chained PASSED               [ 40%]
  tests\test_importer.py::test_validation_chaining PASSED                  [ 46%]
  tests\test_importer.py::test_tolerant_import PASSED                      [ 53%]
  tests\test_importer.py::test_strict_import_keeps_previous_output PASSED  [ 60%]
  tests\test_importer.py::test_formats PASSED                              [ 66%]
  tests\test_importer.py::test_atomic_write_rollback PASSED                [ 73%]
  tests\test_importer.py::test_gzip_and_jsonl_export PASSED                [ 80%]
  tests\test_importer.py::test_transaction PASSED                          [ 86%]
  tests\test_importer.py::test_retry PASSED                                [ 93%]
  tests\test_importer.py::test_generator_and_main PASSED                   [100%]

  ============================== 15 passed in 0.14s ==============================
```

Рисунок 10 — Результат виконання тестів pytest

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> mypy src
  Success: no issues found in 13 source files
```

Рисунок 11 — mypy src у strict-режимі

Результати experiment strict/tolerant

Вхідний файл: 1000 valid і 10 invalid записів, розміщених випадково.

Параметр	Strict	Tolerant
Оброблено	7	1010

Valid	6	1000
Invalid	1	10
Exported	0	1000
Завершення	stop, рядок 8	успішно

Strict mode доречний, коли дані мають бути повністю коректними і частковий результат небезпечний: фінансові операції, офіційні протоколи змагань, міграції БД. Tolerant mode доречний для масового імпорту з зовнішніх джерел, де краще обробити все коректне, а помилки зібрати в окремий звіт для виправлення.

Результати streaming/eager experiment

Records	Eager time, s	Streaming time, s
10 000	0.043	0.025
100 000	0.313	0.330
500 000	2.642	1.221

Таблиця peak memory

Records	Eager peak RAM, MB	Streaming peak RAM, MB
10 000	5.04	0.062
100 000	50.18	0.062
500 000	250.99	0.062

Пам'ять eager-варіанта зростає лінійно (≈ 5 MB на 10 000 записів і ≈ 250 MB на 500 000), streaming тримає сталі ≈ 0,06 MB. За часом різниця на малих наборах незначна (на 100 000 streaming навіть трохи повільніший), а на 500 000 streaming швидший, бо не витрачає час на виділення пам'яті під великий список. Streaming дає суттєву перевагу в пам'яті, але не гарантує мінімальний час виконання.

Експерименти 3–5

- **Exception chaining:** traceback показує ValueError → «The above exception was the direct cause» → RecordValidationError з номером рядка і полем.
- **Atomic output:** під час export навмисно піднято ExportError після 3 записів; попередній valid output не пошкоджено, файл містить коректний JSON з 10 записів, тимчасових .tmp файлів не залишилось, подію зареєстровано в log.

- **Configuration errors:** відсутній config.yaml і неправильний YAML дають ConfigurationError з причиною FileNotFoundError/ParserError; відсутність секції input і result.max = 150 дають ConfigurationError; відсутній input file — DataImportError; недоступний output directory — ExportError. У жодному випадку застосунок не падає з необробленим traceback: main повертає код 2 для конфігурації та 1 для даних.

Результат git status та git log

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Рисунок 12 — Результат git status

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_05> git log --oneline -12
5e58706 (HEAD -> main, origin/main, origin/HEAD) Improve strict mode statistics and log format
68d4767 Add README for Lab_05
0068ef0 Add tests for import and export
2759eea Add reliability experiments
d2d3b6a Add import service and application entry point
4a91bd7 Add atomic write, file transaction and exporters
15425d6 Add streaming importers and validation policy
e547c1b Add configuration loader and logging setup
827954f Add Lab_05 structure, config, dataset and exception hierarchy
76ca0ba Add Lab_04 report
6702ce3 Add README for Lab_04
a010593 Add tests for object model
```

Рисунок 13 — Результат git log --oneline (останні 12 комітів)

Відповіді на контрольні запитання

1. Що таке exception?

Об'єкт, що сигналізує про помилку або виняткову ситуацію під час виконання і перериває звичайний потік програми.

2. Для чого використовується try?

Для позначення блоку коду, у якому може виникнути виняток, що буде оброблено.

3. Для чого except?

Для перехоплення й обробки винятків певного типу, що виникли в блоці try.

4. Коли виконується else після try?

Лише якщо в блоці try не виникло жодного винятку.

5. Коли виконується finally?

Завжди: після try/except/else, навіть при return чи необробленому винятку — для звільнення ресурсів.

6. Для чого використовується raise?

Для явного генерування винятку.

7. Що означає raise без аргументів?

Повторне піднімання поточного винятку всередині except зі збереженням traceback.

8. Чому не рекомендується except: pass?

Він мовчки ховає всі помилки, включно з KeyboardInterrupt і програмними помилками, тому проблеми стають непомітними.

9. Чому потрібно перехоплювати конкретні exception types?

Щоб обробляти лише очікувані ситуації, а несподівані помилки не маскувати і бачити їх.

10. Що таке exception hierarchy?

Ієрархія класів винятків через наслідування; except базового класу перехоплює й усі підкласи.

11. Для чого створювати custom exceptions?

Щоб виражати помилки предметної області, додавати контекст (рядок, поле) і перехоплювати помилки застосунку окремо від системних.

12. Що таке exception chaining?

Зв'язування нового винятку з причиною: оригінальний виняток зберігається в __cause__ або __context__.

13. Що означає raise ... from ...?

Явне вказання причини: новий виняток піднімається, а вказаний стає його __cause__.

14. Яку перевагу дає chaining для debugging?

Traceback показує і високорівневий контекст, і першопричину, тож легко знайти справжнє джерело помилки.

15. Що означає EAFP?

Easier to Ask Forgiveness than Permission — спочатку виконати дію, а помилку обробити в except.

16. Що означає LBYL?

Look Before You Leap — спочатку перевірити умови, а потім виконувати дію.

17. Що таке fail-fast?

Принцип негайно зупинятися з чіткою помилкою, щойно виявлено некоректний стан.

18. Де доцільно застосовувати fail-fast?
При перевірці конфігурації, обов'язкових ресурсів і в strict mode — щоб не обробляти дані з неправильними налаштуваннями.

19. Що таке context manager?
Об'єкт, що керує ресурсом у блоці with: готує його на вході й гарантовано звільняє на виході.

20. Для чого використовується with?
Для безпечної роботи з ресурсами (файли, транзакції) з автоматичним закриттям навіть при помилці.

21. Які methods утворюють context manager protocol?
__enter__() і __exit__(exc_type, exc, traceback).

22. Що робить __enter__()?
Підготовлює ресурс і повертає об'єкт, що присвоюється змінній після as.

23. Що робить __exit__()?
Звільняє ресурс; отримує інформацію про виняток і може його приховати, повернувши True.

24. Чи закриває with файл при exception?
Так, __exit__ викликається завжди, тож файл закривається і при винятку.

25. Для чого contextlib.contextmanager?
Для створення context manager з generator function без написання класу.

26. Яку роль має yield у generator-based context manager?
Код до yield — це __enter__, значення yield передається в as, код після yield (або у finally/except) — __exit__.

27. Що таке pathlib.Path?
Об'єктне представлення шляху до файлу з методами для роботи з файлами та каталогами.

28. Чому pathlib кращий за ручну конкатенацію шляхів?
Він кросплатформений (правильні розділювачі), читабельніший (оператор /) і має зручні методи exists, suffix, mkdir тощо.

29. Для чого задається encoding="utf-8"?
Щоб текст читався і записувався однаково на всіх ОС; без цього Windows може використати іншу кодову сторінку і зіпсувати кирилицю.

30. Що таке CSV?
Текстовий табличний формат, де значення розділені комами, а записи — рядками.

31. Для чого csv.reader?
Для читання CSV як послідовності списків значень.

32. Для чого csv.DictReader?
Для читання CSV як словників, де ключі — назви колонок із заголовка.

33. Чим DictReader зручний для structured data?
Доступ до полів за назвами робить код читабельним і стійким до зміни порядку колонок.

34. Для чого csv.DictWriter?
Для запису словників у CSV з заданим набором колонок і заголовком.

35. Що таке JSON?
Текстовий формат обміну даними з об'єктами, масивами, рядками, числами, true/false/null.

36. Що таке serialization?
Перетворення об'єкта в формат для збереження або передачі (рядок, байти).

37. Що таке deserialization?
Відновлення об'єкта з серіалізованого представлення.

38. Чим json.dump() відрізняється від json.dumps()?
dump записує у файл, dumps повертає рядок.

39. Чим json.load() відрізняється від json.loads()?
load читає з файлу, loads — з рядка.

40. Що таке JSONDecodeError?
Виняток (підклас ValueError), що виникає при розборі некоректного JSON.

41. Для чого використовується YAML?
Для людиночитаних конфігураційних файлів і структурованих даних з відступами.

42. Чому рекомендується yaml.safe_load()?
Він створює лише базові типи; yaml.load з небезпечним loader може виконати довільний код з файлу.

43. Які типові сценарії використання JSON?
REST API, обмін даними між сервісами, експорт результатів, збереження стану.

44. Які типові сценарії використання YAML?
Конфігурації застосунків, CI/CD (GitHub Actions), Docker Compose, Kubernetes.

45. Що таке configuration file?

Зовнішній файл з налаштуваннями програми (шляхи, режими, пороги), що змінюються без зміни коду.

46. Чому configuration краще не hard-code у Python?

Щоб змінювати поведінку без редагування коду, мати різні налаштування для середовищ і не зберігати секрети в коді.

47. Що потрібно перевіряти у configuration?

Наявність обов'язкових секцій і ключів, типи, допустимі діапазони та значення, версію схеми, існування шляхів.

48. Що таке logging?

Стандартний механізм запису подій програми з рівнями важливості, форматуванням і різними виводами.

49. Чим logging відрізняється від print()?

logging має рівні, час, джерело, кілька handlers (файл, консоль), ротацію і вмикається/вимикається конфігурацією.

50. Які існують logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?

Для запису звичайних подій роботи (старт, завершення, прогрес).

52. Для чого logger.warning()?

Для неочікуваних, але некритичних ситуацій (пропущений некоректний запис).

53. Для чого logger.error()?

Для помилок, через які операцію не виконано.

54. Для чого logger.exception()?

Для запису помилки рівня ERROR разом із traceback всередині except.

55. Що містить traceback?

Ланцюжок викликів (файли, рядки, функції) до місця винятку, тип і повідомлення винятку, а також причини при chaining.

56. Чому secrets не можна записувати в logs?

Логи часто доступні багатьом людям і системам, зберігаються довго, тому витік паролів чи токенів через них небезпечний.

57. Що таке streaming I/O?

Читання або запис даних частинами (по рядку/запису) без завантаження всього обсягу.

58. Яка перевага streaming для великих files?

Стала й мала витрата пам'яті та можливість обробити файли, більші за RAM.

59. Що таке eager file loading?

Завантаження всього вмісту файлу в пам'ять одразу (read(), list(reader)).

60. Чому list(reader) може бути небажаним?

Він матеріалізує всі записи, і для великого файлу пам'ять зростає пропорційно розміру (≈ 250 MB на 500 000 записів).

61. Що таке strict validation mode?

Режим, у якому перша некоректна помилка зупиняє весь імпорт.

62. Що таке tolerant mode?

Режим, у якому некоректні записи пропускаються й реєструються, а коректні обробляються.

63. Коли доцільно використовувати strict mode?

Коли важлива повна цілісність: фінанси, офіційні результати, міграції, де частковий імпорт неприпустимий.

64. Коли tolerant mode?

При масовому імпорті з зовнішніх джерел, де допустимо пропустити окремі помилки і виправити їх за error report.

65. Що таке atomic write?

Запис, після якого файл або повністю новий, або залишається старим, без проміжного пошкодженого стану.

66. Для чого використовується temporary file?

Щоб писати дані окремо від цільового файлу і замінити його лише після успішного завершення.

67. Що може статися при звичайному output write та аварійному завершенні?

Файл залишиться частково записаним або порожнім, а попередні коректні дані буде втрачено.

68. Навіщо видаляти temporary file після error?

Щоб не залишати сміття і частково записані файли, які можна сплутати з результатом.

69. Що таке JSON Lines?

Формат, де кожен рядок файлу — окремий JSON-об'єкт.

70. Чому JSON Lines зручний для streaming?

Кожен рядок читається і записується незалежно, тому файл можна обробляти по одному запису й дописувати в кінець.

Висновок

У лабораторній роботі створено надійний модуль імпорту та експорту спортивних результатів. Конфігурація завантажується з config.yaml через yaml.safe_load, перевіряється (schema_version, обов'язкові секції, діапазон result, режим) і може бути перевизначена змінними середовища. Дані читаються потоково через Importer Protocol (CSV, JSON, JSON Lines, YAML), перевіряються ValidationPolicy і обробляються в strict або tolerant режимі зі статистикою, окремим error report і summary. Власна exception hierarchy з exception chaining дає точні повідомлення з першопричиною, logging з ротацією фіксує всі події, а atomic_write і FileTransaction гарантують, що збій не пошкодить існуючий output. Експерименти підтвердили, що streaming тримає сталу пам'ять ≈ 0,06 MB проти 250 MB у eager-варіанті на 500 000 записів, strict mode зупиняється на першій помилці без пошкодження даних, а всі помилки конфігурації обробляються контрольовано з кодами завершення. Код перевірено mypy у strict-режимі та 15 тестами pytest.